

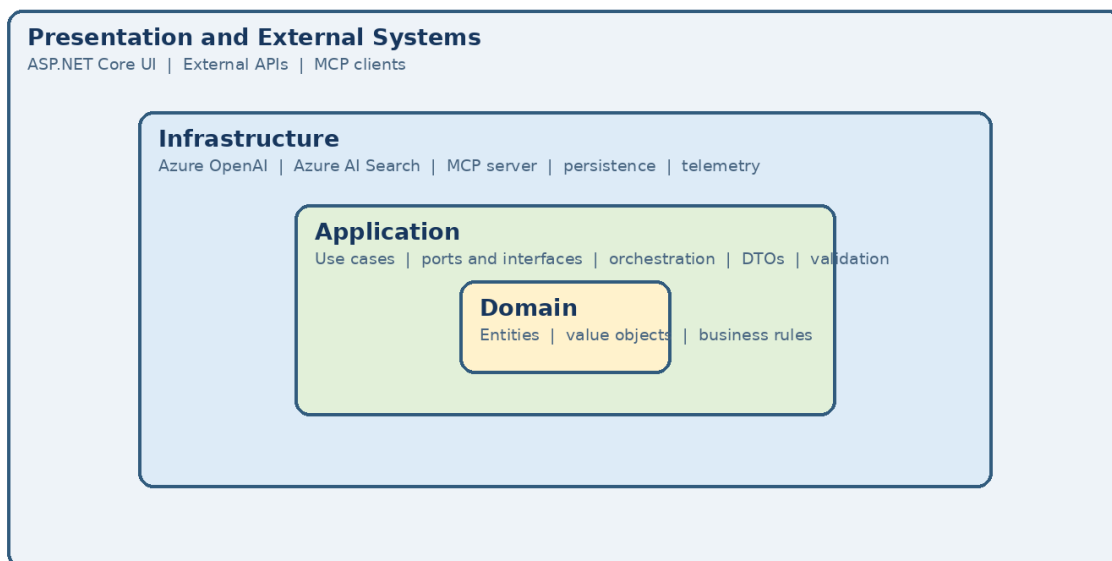
GenAI Powered .NET Application Development

Use Case 1 Enterprise IT Service Desk Copilot

Standalone professional capstone assignment for associates with six months of experience

Training assignment specification

Required Clean Architecture



Source-code dependencies point inward; runtime calls travel outward through interfaces

Prepared from the supplied NET GenAI Training Curriculum

Purpose and expected outcome

This assignment defines one complete GenAI-enabled .NET capstone solution: Enterprise IT Service Desk Copilot. The team will implement the project through progressive increments covering prompt engineering, Azure OpenAI, RAG, function calling, Semantic Kernel or equivalent orchestration, bounded agents, MCP, security, evaluation, monitoring and deployment.

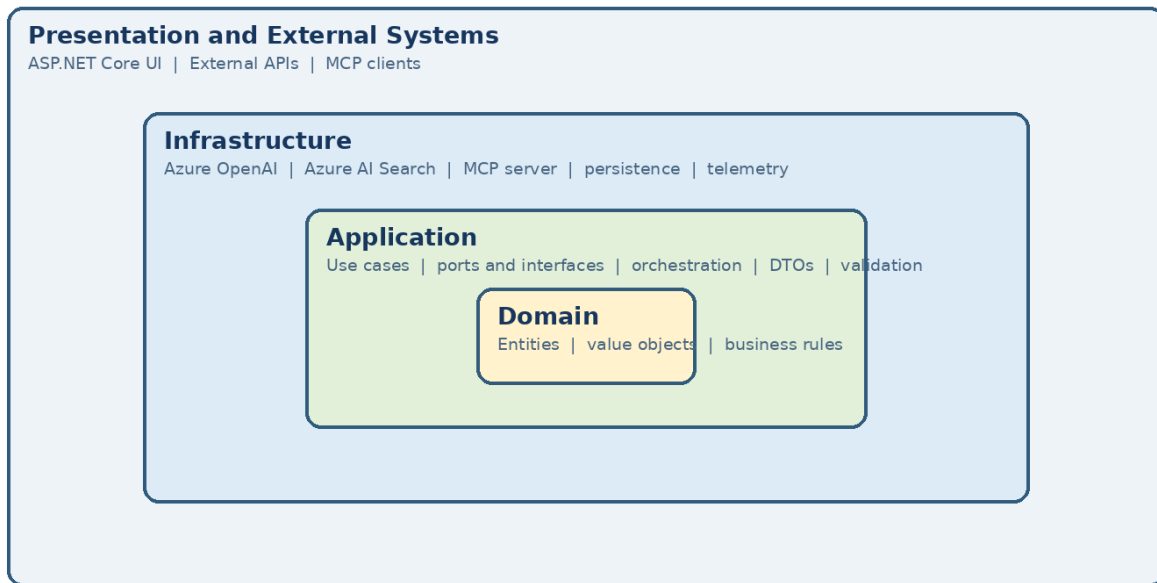
The difficulty is calibrated for developers with approximately six months of experience. Deterministic business rules are explicit, integrations may begin with stubs, architecture boundaries are prescribed, and stretch goals are separated from the mandatory path.

How to use this pack

1. Trainer assigns this use case to a team of three or four associates.
2. The team creates a backlog from the requirements and completes the three-day sprint plan.
3. Team uses GitHub Copilot as an assistant but reviews, tests and documents every accepted change.
4. Team demonstrates both success and failure paths, including prompt injection, tool failure and denied write actions.
5. Trainer scores the submission using the common rubric at the end of this document.

Common Clean Architecture requirements

Required Clean Architecture



Source-code dependencies point inward; runtime calls travel outward through interfaces

Figure 1 Required dependency direction

- Domain contains business meaning and deterministic rules only. It must not reference Azure SDKs, Semantic Kernel, ASP.NET Core, databases or logging frameworks.
- Application coordinates use cases and owns interfaces for model, retrieval, tools, persistence, approval and telemetry.
- Infrastructure implements those interfaces and contains Azure OpenAI, Azure AI Search, MCP, external API and database adapters.
- Web maps authenticated requests to Application commands and queries, streams results and displays safe errors; it contains no business rule.
- All external calls support cancellation, timeouts and correlation. Write actions require authorization, validation and human approval.
- Prompts are versioned assets. Their output is treated as untrusted until schema and business validation succeed.

Required technical baseline

Area	Minimum expectation
Platform	.NET 8 or the organization-approved current LTS; ASP.NET Core web API or web app
AI	Approved Azure OpenAI chat deployment plus embedding deployment
Retrieval	Azure AI Search index with source, version and citation metadata
Orchestration	Semantic Kernel plugins/function calling plus an agent framework or AutoGen-style workflow
Interoperability	One MCP server exposing typed, scoped enterprise tools
Quality	NUnit tests, integration tests and a repeatable prompt-evaluation dataset
Operations	Structured logging, OpenTelemetry/Application Insights, token and latency measurement
Security	Managed identity or safe local credential chain, role-based authorization and secret-free repository
Delivery	Cloud deployment with configuration separated from code and a documented rollback/fallback

Use Case 1 Enterprise IT Service Desk Copilot

A grounded support assistant that answers policy questions, diagnoses common incidents and creates approved tickets.

Attribute	Definition
Business domain	Internal IT operations and employee support
Target user	Employees, service-desk analysts and support managers
Delivery mode	Three-day team sprint with an end-to-end demonstration
Recommended team	3-4 associates with approximately six months of .NET experience
Complexity	Foundation-to-intermediate; core path is mandatory and stretch work is optional

Business context and problem

Employees search multiple knowledge articles before reporting a problem, while analysts repeatedly ask for missing diagnostic details. The proposed assistant must provide evidence-backed guidance, collect structured facts, call approved support tools and hand unresolved work to a human without inventing status or policy.

Business objective

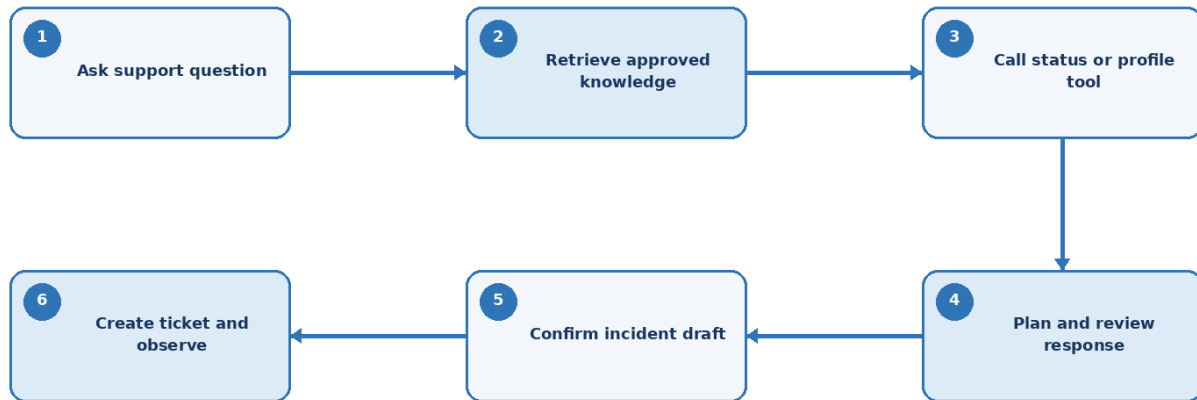
- Reduce time spent locating approved troubleshooting steps.
- Improve completeness and consistency of incident tickets.
- Demonstrate a safe read-to-write agent workflow with human approval.
- Produce measurable evidence for groundedness, latency, token consumption and escalation quality.

What the team will build

Build an ASP.NET Core support portal with streaming chat. The assistant searches IT policy and troubleshooting documents, checks system status, proposes diagnostic steps, and prepares an incident. A reviewer agent checks grounding and safety before the user confirms ticket creation.

Increment t	Capability added	What it demonstrates
1	Prompt and chat completion	Model connectivity and prompt constraints
2	RAG with citations	Grounded enterprise answers
3	Status and user-profile tools	Function calling and data retrieval
4	Incident draft plus approval	Controlled business write action
5	Planner, specialist and reviewer	Multi-agent orchestration and evaluation
6	Cloud deployment and telemetry	Enterprise operations

Service Desk Request Flow



Every model response is grounded, validated, logged and subject to authorization before a write action is executed.

Figure 2 Enterprise IT Service Desk Copilot end-to-end flow

Scope

In scope	Out of scope
IT policy and knowledge search; status lookup; guided diagnosis; incident draft and approval; session history; citations; evaluation and telemetry	remote device control; password reset execution; production identity administration; free-form database access; fully autonomous ticket submission

Personas and responsibilities

Persona	Need	Permitted actions
Employee	Fast, trustworthy help	Search knowledge; run read-only status checks; approve own ticket draft
Analyst	Complete diagnostic context	Review and update assigned drafts; add resolution feedback
Manager	Quality and usage visibility	View aggregated evaluation, latency and cost metrics
Administrator	Safe configuration	Manage indexes, prompts and tool allow-lists; cannot alter audit history

Functional requirements

ID	Requirement	Priority
FR-01	Accept a support question and stream a response with a correlation ID.	Must
FR-02	Classify the request as policy, how-to, outage, incident or unsupported.	Must
FR-03	Retrieve relevant approved articles using hybrid or semantic search and cite sources.	Must

ID	Requirement	Priority
FR-04	Ask only for missing diagnostic information and preserve session context.	Must
FR-05	Call a read-only system-status function when current status is required.	Must
FR-06	Create a structured incident draft with summary, impact, urgency, category and evidence.	Must
FR-07	Require explicit user confirmation before the ticket API is called.	Must
FR-08	Use Planner, Support Specialist and Reviewer roles for a complex request.	Should
FR-09	Expose knowledge search and incident draft operations through an MCP server.	Should
FR-10	Capture feedback and evaluate groundedness, relevance and safe refusal.	Must

Business rules

Rule	Enforcement
BR-01	Only approved and active knowledge sources may be retrieved.
BR-02	An outage claim requires status-tool evidence; otherwise label it unconfirmed.
BR-03	Priority is computed by deterministic impact and urgency rules, never solely by the model.
BR-04	A ticket cannot be submitted until the authenticated user confirms the final draft.
BR-05	Credentials, passwords, access tokens and secret values must be rejected and never logged.
BR-06	Unsupported or low-confidence cases are routed to an analyst with the conversation summary.

Detailed scenarios

Scenario 1 Grounded policy answer

Field	Specification
Primary actor	Employee
Precondition	User is authenticated; approved policy documents are indexed.
Expected result	Answer is accurate, cites an approved source and contains no unsupported policy statement.

Main flow

1. User asks whether a personally owned device may connect to the corporate network.
2. The application validates the request and retrieves the most relevant policy chunks.
3. The AI composes a concise answer using only retrieved evidence.
4. The UI streams the response and displays source title and section.

Alternate and failure flow

- If retrieval confidence is below threshold, the assistant says it cannot verify the policy and offers escalation.
- If the prompt contains instructions copied from a document, those instructions are ignored as untrusted content.

Scenario 2 Diagnose a common issue

Field	Specification
Primary actor	Employee
Precondition	Troubleshooting articles and system-status tool are available.
Expected result	The guidance is grounded, sequenced, safe and records the resolution outcome.

Main flow

1. User reports that VPN login fails.
2. Assistant asks for error code, device type and whether the issue affects other services.
3. The status tool checks current VPN health.
4. RAG supplies approved steps; the assistant presents them one at a time.
5. User confirms whether the issue is resolved.

Alternate and failure flow

- A tool timeout produces a clearly marked status-unavailable message.
- Unsafe steps such as disabling security controls are refused.

Scenario 3 Create an approved incident

Field	Specification
Primary actor	Employee
Precondition	Diagnosis did not resolve the problem; ticket service is available.
Expected result	Exactly one ticket is created after approval and the audit log links it to the correlation ID.

Main flow

1. Assistant generates a schema-valid incident draft.
2. Application calculates priority from deterministic rules.
3. Reviewer agent checks for unsupported claims and sensitive data.
4. User edits or confirms the draft.
5. Ticket adapter submits the approved request and returns the real ticket number.

Alternate and failure flow

- Rejection returns to draft state without calling the ticket service.
- A failed ticket call returns a retryable error and never invents an ID.

Scenario 4 Complex issue agent collaboration

Field	Specification
Primary actor	Analyst
Precondition	Multi-agent workflow is enabled.
Expected result	Agent output includes evidence and finishes within configured step and token limits.

Main flow

1. Planner decomposes the issue into knowledge search, live status and ticket-history tasks.
2. Specialist executes only allow-listed read tools.
3. Reviewer verifies evidence and identifies missing facts.
4. Application returns a consolidated recommendation and proposed next action.

Alternate and failure flow

- Loop count and token budget stop excessive agent iteration.
- Any proposed write action pauses for human approval.

Clean Architecture solution design

The solution must keep domain and application policy independent of ASP.NET Core, Azure SDKs, search technology, databases and agent frameworks. Infrastructure components implement interfaces owned by the Application layer and are registered only in the composition root.

Layer	Required contents	Dependency rule
Domain	Entities, value objects, domain services, policies and domain exceptions	References no other project
Application	Commands, queries, use-case handlers, DTOs, validators and interface ports	Depends only on Domain
Infrastructure	Azure OpenAI, AI Search, database, MCP, identity and telemetry adapters	Implements Application interfaces
Web	ASP.NET Core endpoints, streaming UI, authentication and request mapping	Calls Application; no business rules
Workers	Indexing, long-running work and agent workflow hosts	Calls Application through defined ports
Tests	Domain, application, contract, integration, prompt evaluation and end-to-end tests	Mirrors production boundaries

Suggested solution structure

Repository layout

```
ServiceDeskCopilot.sln
src/
  ServiceDesk.Domain/
    Incidents/ Policies/ Common/
  ServiceDesk.Application/
    Chat/ Knowledge/ Incidents/ Abstractions/ Validation/
  ServiceDesk.Infrastructure/
    AzureOpenAI/ Search/ Ticketing/ Mcp/ Persistence/ Telemetry/
  ServiceDesk.Web/
    Endpoints/ Streaming/ Auth/ Composition/
  ServiceDesk.Worker/
    Indexing/ AgentWorkflows/
tests/
  Domain.Tests/ Application.Tests/ Integration.Tests/ PromptEval.Tests/
```

Core application interfaces

Interface	Purpose	Example implementation
IChatModel	Complete and stream model responses	Azure OpenAI adapter
IKnowledgeRetriever	Return ranked chunks and citations	Azure AI Search adapter
ISystemStatusReader	Read live service health	Status API or MCP tool
IIncidentGateway	Create an approved ticket	Service-management API
IAgentWorkflow	Coordinate plan, execute and review	SK or agent-framework adapter
IAiTelemetry	Record safe AI measurements	OpenTelemetry/Application Insights

Data and integration design

Source or component	Sample content	Access pattern
Knowledge index	VPN, email, device and security articles with source metadata	Hybrid/semantic retrieval
Status API	Service name, state, started time and message	Read-only function call
Ticket API	Draft and confirmed incident payloads	Write after approval
Session store	Messages, cited sources and state, with retention limit	Repository interface
MCP server	search_knowledge, get_status, create_incident_draft	Typed allow-listed tools

Prompt engineering requirements

- Create a reusable system prompt containing role, permitted scope, grounding rule, citation rule, refusal behavior and output schema.
- Demonstrate zero-shot and few-shot versions for one task, compare the outputs and record why the selected version is more reliable.
- Use the Ask-Review-Improve and Explain-Generate-Refine patterns during development; maintain prompts as versioned application assets.
- Decompose complex work into explicit steps but request only a concise rationale or evidence, not hidden chain-of-thought.
- State assumptions, constrain formats, validate JSON outputs and reject an answer when required evidence is absent.

Minimum prompt contract

ROLE: Internal IT support assistant.
 CONTEXT: Use only SOURCES and TOOL_RESULTS supplied by the application.
 RULES: Cite factual guidance. Never request or repeat secrets. Never claim a ticket exists unless the create tool returned an ID.
 TASK: Classify the request, ask for missing facts, then answer or propose escalation.
 OUTPUT: Valid JSON matching SupportReply { category, answer, citations[], proposedAction, confidence }.
 FALLBACK: If evidence is insufficient, say what is missing and route to a human.

Agent and tool design

Component	Responsibility	Safety boundary
Planner	Selects knowledge, status and history tasks	No direct external access
Support Specialist	Uses retrieval and read-only tools	Tool allow-list and argument validation
Reviewer	Checks grounding, privacy and completeness	Cannot execute tools
Application handler	Applies authorization, rules and approval state	Only component that permits write adapter

Implementation note: Semantic Kernel may be used for plugin registration and function calling. An agent framework or AutoGen-style orchestration may coordinate Planner, Specialist and Reviewer roles. MCP exposes only approved read or write tools; the domain rules remain inside the application, not inside prompts.

Security, responsible AI and operations

Area	Mandatory control	Evidence
Identity	Use managed identity or environment-based developer credentials; never commit secrets	Configuration scan and successful role-based access
Authorization	Enforce role checks in application handlers and again at write adapters	Denied-action test
Prompt injection	Treat retrieved content as data, delimit it, block instruction override and allow-list tools	Adversarial test set
Privacy	Redact or mask sensitive fields before model calls and logs	Redaction test and log sample
Validation	Validate input length, structured output, citations and tool arguments	Automated validation tests
Observability	Record correlation ID, latency, token usage, retrieval hits, tool calls and failures	Dashboard or structured log evidence
Resilience	Timeout, retry with backoff for transient faults, circuit breaker and user-safe fallback	Simulated 429 and dependency outage
Human review	Require approval for consequential write actions	Approval and rejection audit entries

Sprint implementation plan

Day	Focus	Expected checkpoint
1	Plan the use case; create solution; model the domain; connect Azure OpenAI; build one chat endpoint; use Copilot for scaffolding and document all accepted changes	Compiling vertical slice with tests and safe configuration
2	Add document ingestion, embeddings, Azure AI Search, grounded RAG, citations, Semantic Kernel plugins or equivalent tool calls, and MCP integration	Search and tool calls work through Application interfaces
3	Add agents, approval flow, streaming, security tests, evaluation set, telemetry, deployment configuration and demo documentation	Deployed end-to-end solution with quality report

Testing and evaluation

Test category	Minimum cases	Pass condition
Domain unit	At least 8 business-rule and boundary tests	All deterministic rules pass without external services
Application unit	At least 6 handler tests with mocked ports	Success, denial, validation and fallback paths pass
Integration	Chat, embeddings/search, tool adapter and MCP contract	Approved test environment returns schema-valid data
Prompt evaluation	At least 15 golden questions plus 5 adversarial prompts	Meets thresholds for groundedness, relevance and refusal
API	Authentication, validation, streaming, cancellation and error contract	No internal exception or secret is exposed
Performance	Five concurrent users and one long-running request	Target latency recorded; no unhandled failures

Definition of done

- Solution builds from a clean checkout and automated tests pass.
- No API key, connection string, personal data or prompt transcript is committed to source control.
- Every answer based on enterprise documents shows source title and section or page metadata.
- The application declines unsupported questions and never invents a successful write action.
- One read tool and one controlled write tool work through interfaces; the write requires authorization and approval.
- Streaming, cancellation, loading and safe error states are visible in the web experience.
- Logs demonstrate correlation, latency, tokens, retrieval, agent steps and tool outcomes without sensitive content.
- README includes architecture, setup, model/search configuration, test commands, deployment and known limitations.

Required demonstration script

1. Ask a policy question and open its citation.
2. Ask about a known service outage and show the status function call.

3. Submit a prompt-injection attempt embedded in retrieved text and show safe handling.
4. Diagnose a VPN failure and create an incident draft.
5. Reject the draft, then approve a corrected version and display the real ticket ID.
6. Open telemetry and the evaluation summary for the same correlation ID.

Starter data and assumptions

- Provide 8-12 short IT knowledge documents in PDF, JSON or text format.
- Use a fake status API with at least three services and deterministic states.
- Use an in-memory or stub ticket adapter before connecting a real sandbox API.
- Supply 20 evaluation prompts: 10 answerable, 5 insufficient-evidence and 5 adversarial.

Stretch goals

- Add conversation summarization for long sessions.
- Use managed identity for model, search and storage access.
- Add analyst feedback to an offline prompt-evaluation pipeline.
- Deploy a background indexing worker and blue-green prompt versioning.

Common assessment rubric

Criterion	Marks	Full-credit evidence
Business fit and completeness	10	Mandatory scenarios work end to end; scope and user roles are clear.
Clean Architecture	15	Dependencies point inward; domain rules are deterministic; adapters are replaceable and tested.
Prompt engineering	10	Prompts are clear, constrained, reusable, versioned and evaluated; outputs are validated.
RAG and grounding	15	Search uses metadata filters; answers cite approved sources; insufficient evidence is handled safely.
Tools, MCP and agents	15	Typed tools work; agents are bounded; write actions require authorization and approval.
Security and responsible AI	10	Secrets, privacy, injection, authorization and safe-failure controls are demonstrated.
Testing and evaluation	10	Unit, integration, API, prompt and adversarial tests are repeatable and meaningful.
Operations and deployment	10	Deployed solution includes telemetry, token/cost tracking, resilience and health evidence.
Code quality and demonstration	5	Readable code, useful documentation, clear walkthrough and honest limitations.

Submission checklist

- Source repository with clean commit history and no secrets.
- Architecture diagram and short architecture-decision record.
- Running application plus deployment URL or recorded local demonstration if cloud access is restricted.
- Sample data, index schema and reproducible ingestion instructions.
- Prompt assets, evaluation dataset and result summary.
- Automated test report and screenshots or traces for critical failure paths.
- README with prerequisites, configuration, run, test, deploy and troubleshooting instructions.
- Five-minute presentation explaining business value, clean architecture, safety controls and known limitations.

Trainer acceptance gate

A submission cannot receive a passing score if it commits a secret, fabricates a tool result, performs a consequential write without approval, exposes data across users, or cannot show the source of a grounded enterprise answer. These are release blockers even when the application otherwise appears functional.